

A Comparative Study of LRU Cache Storage Strategies and Singly Linked List Deallocation

Arpan Pathak

11 September 2026

Abstract

Two experiments are reported. The first compares two storage strategies for a fixed-capacity least-recently-used cache: an index arena, in which nodes are held in a contiguous vector and linked by integer offsets, and a reference-counted doubly linked list, in which each node is a separate heap allocation linked by pointers. On a workload of ten million requests with a measured hit rate of 89.8 percent, the arena completed in 1.00 s against 1.86 s for the reference-counted list, a ratio of 1.87. The magnitude of the difference is bounded by the hash table lookup that both implementations perform on every request; across access patterns the ratio ranged from 1.1 to 1.9.

The second experiment compares four singly linked list variants that combine two node representations with two deallocation strategies. The variants that rely on compiler-generated deallocation cannot free more than approximately 265,000 nodes on an 8 MiB stack, because freeing a node frees its successor and the recursion consumes one stack frame per element; beyond that size the process terminates with a stack overflow. The iterative variants free five million nodes without incident. The iterative strategy is not substantially faster, saving about 0.3 ms on a 3.2 ms deallocation at 200,000 nodes; its principal contribution is the removal of the size limit.

The measurement procedure, including the treatment of allocation warm-up, and the threats to the validity of both results are discussed.

Keywords: cache replacement, index arena, reference counting, linked list, stack overflow, benchmarking methodology

1. Introduction

A cache that must bound memory occupancy needs a replacement policy, and the least-recently-used policy requires a recency order over the stored entries. The order is conventionally maintained by a doubly linked list, and the two decisions that determine the cost of that list are how nodes are allocated and how links between them are represented.

The first decision concerns ownership. A node may be owned individually, as one heap allocation reached through a pointer, or collectively, as one element of a contiguous array. Individually owned nodes support stable addresses and permit removal without disturbing the rest of the structure, at the cost of one allocation per node and a reference count or equivalent mechanism to express ownership. Collectively owned nodes require

the allocator to be invoked once, and express the identity of a node by its index, at the cost of a level of indirection.

The second decision concerns deallocation. A recursively defined structure releases its storage through a recursive destructor unless the recursion is replaced by an iterative walk. This choice is frequently treated as a matter of performance, and the present study finds that the performance difference is small. Its consequences are otherwise: the recursive form has a maximum size beyond which the program terminates.

This document reports measurements of both decisions. Section 2 describes the structures. Section 3 gives the experimental method. Section 4 presents the results, Section 5 discusses them and states the threats to their validity, and Section 6 concludes.

2. Background

2.1 Storage for a fixed-capacity LRU cache

Both cache implementations under study maintain a hash table that maps a key to a node, and a doubly linked list that orders the nodes by recency. Insertion of a new entry when the cache is full removes the least recently used node. A lookup that succeeds moves the located node to the most recently used end.

In the index arena implementation, the nodes are elements of a `Vec`, and the links are values of type `Option<usize>`. The hash table maps a key to the index of its node. Removal of the least recently used entry reuses that entry's slot rather than deallocating it, so the arena is allocated once, grows to its capacity, and is never resized during operation. The implementation is `benchmarking_examples/cache/arena.rs`.

In the second implementation, each node is allocated separately and reached through a reference-counted cell. Forward links hold strong references and backward links hold weak references, which prevents ownership cycles. Removal of the least recently used entry removes the tail node from the hash table and detaches it, after which the new entry is allocated as a fresh node. The implementation is `benchmarking_examples/cache/rc_list.rs`.

Both implementations satisfy the same trait, `cache::Cache`, and are therefore exercised by one workload.

2.2 Node representation and deallocation in a singly linked list

Four variants of a singly linked list are compared. They share push and pop operations and differ in two independent properties.

VARIANT	NODE REPRESENTATION	DEALLOCATION
<code>box</code>	<code>Option<Box<Node<T>>></code>	compiler-generated, recursive
<code>enum</code>	<code>enum ListNode<T></code>	compiler-generated, recursive
<code>box+drop</code>	<code>Option<Box<Node<T>>></code>	explicit, iterative
<code>enum+drop</code>	<code>enum ListNode<T></code>	explicit, iterative

The two representations have different sizes. `Option<Box<Node<T>>>` occupies one machine word on a 64-bit target, because a boxed value is never null and the all-zero address is therefore available to encode the empty case. The enum representation carries a discriminant in addition to the boxed continuation, and is consequently larger. The variant definitions are in `benchmarking_examples/lists/`.

The deallocation strategies differ in their recursion. The compiler-generated destructor for a node releases the node, whose destructor releases its successor, and so on, consuming one stack frame per element. The iterative strategy replaces the head of the list with the empty state and then walks the chain, detaching each node before it is freed, so no destructor is called with the remainder of the list still attached.

3. Method

3.1 Environment

PROPERTY	VALUE
Processor	AArch64, 8 cores, NVIDIA Jetson development board
Compiler	<code>rustc 1.96.0-nightly</code>
Build profile	<code>--release</code>
Concurrency	one thread per measurement
Default stack size	8 MiB

Absolute times are properties of this machine and this toolchain. The ratios and the maximum list size are the quantities intended to generalise, the latter only within the constraint of the stack size in force.

3.2 Workloads

Cache workload. A fixed-capacity cache of 65,536 entries receives ten million requests over a key space of 131,072 keys. Four requests in five are drawn from a hot set comprising one percent of the key space, and the remainder are drawn uniformly over the whole key space. The request sequence is generated from a fixed seed, so both implementations receive an identical sequence. A request that misses performs an insertion, which evicts the least recently used entry once the cache has reached capacity. The measured hit rate was 89.8 percent, comprising 8,979,314 hits and 1,020,686 insertions.

Linked list workload. Each variant is measured at 200,000 elements, a size at which all four complete. The two iterative variants are additionally measured at 5,000,000 elements. The recursive variants are measured at incrementally increasing sizes until termination occurs, to establish the size at which deallocation fails.

3.3 Measurement procedure

An initial observation established that a single timing pass is not reproducible on this platform. Timings of one push loop of fixed size varied between 2.3 ms and 11.4 ms

depending on whether the measurement was the first large allocation performed by the process. The variance is attributable to the allocator and the operating system rather than to the structure under test: a first pass over a fresh heap requires pages to be mapped, and that cost is charged to the structure being measured.

Accordingly, each measurement is preceded by a warm-up pass of the same magnitude as the measured pass, and then repeated three times, with the shortest of the three reported. The shortest pass is preferred to the mean because the quantity of interest is the cost of the work, while a lengthened pass reflects interference from outside the process. This procedure follows the practice recommended for benchmarking where the number of repetitions is constrained [3, 4]. The residual variation between runs of the cache experiment, computed over the reported configuration, is approximately three percent.

Measurements are normalised per request for the cache experiment and per element for the list experiment, so that the two experiments can be reported in comparable units.

3.4 Verification and controls

Each measurement is accompanied by a check that the intended work was performed.

In the list experiment, every value inserted is removed and compared with the expected value, so that a structure which loses or reorders elements cannot report a time. The verification loop is not included in any timed interval.

In the cache experiment, the two implementations process the same request sequence and their hit counts are compared before any timing is reported. Agreement establishes that both implementations were given the same problem.

The variants `box` and `box+drop` share an identical push implementation and differ only in deallocation. Their push times therefore serve as a control: a discrepancy beyond the run-to-run variation indicates a fault in the measurement rather than a difference between the structures.

4. Results

4.1 Cache storage strategies

LRU cache: 65,536 entries, 10,000,000 requests over 131,072 keys

hit rate: 89.8% (8,979,314 hits, 1,020,686 inserts)

storage	total	per request
arena (Vec + indices)	1.00s	100 ns
Rc<RefCell> list	1.86s	186 ns

The arena required 1.00 s and the reference-counted list 1.86 s, giving a ratio of 1.87 in favour of the arena. Measured per request, the costs are 100 ns and 186 ns respectively. Repeated runs of the same configuration produced ratios between 1.81 and 1.87.

4.2 Deallocation in singly linked lists

All four variants completed at 260,000 elements. At 270,000 elements the two recursive variants terminated with a stack overflow and the two iterative variants completed.

VARIANT	260,000 ELEMENTS	270,000 ELEMENTS
box	completed	stack overflow
enum	completed	stack overflow
box+drop	completed	completed
enum+drop	completed	completed

The termination message recorded for the recursive variants is:

```
thread 'main' has overflowed its stack
fatal runtime error: stack overflow, aborting
```

On an 8 MiB stack, the limit therefore lies between 260,000 and 270,000 elements, corresponding to approximately 32 bytes of stack per element.

Timings at 200,000 elements, where all four variants complete:

variant	elements	push	drop
-----	-----	-----	-----
box	200,000	2.40ms	3.23ms
enum	200,000	3.42ms	3.20ms
box+drop	200,000	2.32ms	2.91ms
enum+drop	200,000	3.62ms	2.52ms

Timings at 5,000,000 elements, where only the iterative variants complete:

box+drop	5,000,000	58.11ms	72.32ms
enum+drop	5,000,000	90.98ms	67.51ms

Normalised per element at 200,000 elements, insertion costs 12.0 ns for the boxed node and 17.1 ns for the enum node, and deallocation costs 16.2 ns, 16.0 ns, 14.6 ns and 12.6 ns for the four variants in the order listed.

5. Discussion

5.1 Interpretation of the cache result

The arena was faster by a factor of 1.87 on the reported workload, and the mechanism is visible in the operations each implementation performs when a lookup succeeds. The arena writes two indices into vector elements and reads two more; no memory is allocated or released and no reference count is modified. The reference-counted list clones a handle, upgrades a weak handle, borrows a `RefCell`, writes through the borrow, and drops the clone. Each step is small, but all of them occur on every successful lookup, and the node they operate on must be reached through a pointer to a separate allocation.

Eviction differs in the same direction. The arena reuses the least recently used slot. The list frees the tail node and allocates a node for the incoming key, an allocation and a deallocation that the arena does not perform.

The ratio is nevertheless smaller than the difference in list manipulation alone would suggest, because every request performs a hash table lookup and that cost is identical in both implementations. The measured ratio therefore depends on the proportion of each request that is spent on list maintenance. Across access patterns, the ratio ranged from 1.1 to 1.9; a stream with no reuse tends towards the lower end, since such a stream is dominated by lookups that miss and by evictions.

5.2 Interpretation of the list result

The difference in insertion cost between the two node representations, 12.0 ns against 17.1 ns per element, is consistent with the difference in node size. Both representations perform one allocation per element; the enum node additionally carries a discriminant, so each insertion moves more memory.

The difference in deallocation cost between the recursive and the iterative strategies is small, 0.32 ms and 0.68 ms at 200,000 elements, against totals of 3.23 ms and 3.20 ms. The iterative strategy is therefore not a performance measure. Its contribution is that it removes the maximum size: the recursive variants terminate above approximately 265,000 elements on an 8 MiB stack, and the iterative variants complete at 5,000,000, the largest size tested.

5.3 Threats to validity

Internal validity. The measurements are taken on a single machine, with one implementation measured after another in the same process. Allocation warm-up was identified as a source of variation and is controlled by the procedure described in Section 3.3, and the two list variants that share an insertion implementation provide a control on that stage. The deallocation measurements remain the least stable part of the results, since they follow a sequence of allocations and deallocations of the same memory.

External validity. Absolute times are specific to this processor, compiler and allocator. The maximum list size is a property of the stack size in addition to the code; a program running on a thread with a smaller stack would fail at a smaller size, and one with a larger stack at a larger size. The value of 5,000,000 reported for the iterative variants is the largest size tested and not an observed limit.

Construct validity. The cache experiment measures request latency for both a lookup and an insertion, but reports only the aggregate. It does not separate the cost of the hash table lookup from the cost of list maintenance, and it does not report peak memory, although the two node representations differ in size. The list experiment measures insertion, removal and deallocation, and does not measure iteration, search, or insertion at an arbitrary position.

Generality. Both experiments are single-threaded and therefore characterise neither contention nor synchronisation cost. A shared arena would require sharding or locking, and the cost of that mechanism is not represented here.

6. Conclusion

For a fixed-capacity LRU cache, an index arena was faster than a reference-counted linked list by a factor of 1.87 on the reported workload, with the difference attributable to the per-node bookkeeping and the allocation performed on eviction. The ratio is bounded by the hash table lookup common to both implementations, and ranged from 1.1 to 1.9 across access patterns.

For a singly linked list, an iterative deallocation strategy is worth adopting for robustness rather than for speed. It saved about ten percent of the deallocation time at the size measured, and it raised the maximum list size from approximately 265,000 elements to beyond 5,000,000, the largest size tested.

Further work would separate lookup cost from list maintenance cost by timing the two phases independently, report peak memory for each variant, and extend the cache experiment to a multi-threaded configuration in which the synchronisation cost of the arena becomes measurable.

References

1. The Rust Reference. [Type layout](#). On size, alignment, and the null-pointer optimisation.
2. The Rust Standard Library. [std::mem::size_of](#), [std::mem::align_of](#), and [std::thread::Builder::stack_size](#).
3. T. Kalibera and R. Jones. Rigorous Benchmarking in Reasonable Time. *Proceedings of the 2013 International Symposium on Memory Management (ISMM)*, 2013.
4. A. Georges, D. Buytaert, and L. Eeckhout. Statistically Rigorous Java Performance Evaluation. *Proceedings of the 22nd Annual ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*, 2007.
5. T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney. Producing Wrong Data Without Doing Anything Obviously Wrong! *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2009.

Appendix A. Source files

FILE	CONTENTS
<code>benchmarking_examples/cache/mod.rs</code>	The <code>Cache</code> trait
<code>benchmarking_examples/cache/arena.rs</code>	Index arena implementation
<code>benchmarking_examples/cache/rc_list.rs</code>	Reference-counted implementation
<code>benchmarking_examples/lists/mod.rs</code>	The <code>SinglyList</code> trait
<code>benchmarking_examples/lists/boxed.rs</code>	Boxed node, recursive deallocation
<code>benchmarking_examples/lists/enum_node.rs</code>	Enum node, recursive deallocation
<code>benchmarking_examples/lists/boxed_drop.rs</code>	Boxed node, iterative deallocation
<code>benchmarking_examples/lists/enum_drop.rs</code>	Enum node, iterative deallocation
<code>benchmarking_examples/benchmark.rs</code>	The measurement program
<code>rust-interview-lab/BENCHMARKS.md</code>	Captured program output